

THE UNIVERSITY OF ZAMBIA

SCHOOL OF NATURAL AND APPLIED SCIENCES
DEPARTMENT OF COMPUTING AND INFORMATICS

Final Year Project Proposal

Academic Year 2026

Project Title: VERISCREEN: A DIGITAL ACADEMIC DOCUMENT VERIFICATION AND EXCHANGE PLATFORM

Student Names:

- Name: Samuel Chibinji Mwanza Computer Number: 2021465934
- Name: Nosiku Mukuka Computer Number: 2022030346

Supervisor: Mrs Monica Monde Kabemba

Department: Department of Computing and Informatics

Date of Submission: March 16, 2026

Contents

1 Introduction **4**

1.1 Problem Statement 5

1.1.1 Challenge 1: Document Accessibility 5

1.1.2 Challenge 2: Loan Application Verification 5

1.2 Project Aim 6

1.3 Objectives 6

1.4 Scope of the Project 6

1.4.1 In Scope: 6

1.4.2 Out of Scope: 7

2 Literature Overview **8**

3 Proposed Methodology **9**

3.1 Development Approach 9

3.2 System Architecture 9

3.3 Complete Workflow Description 9

3.3.1 Phase 1: Student Document Access (VeriScreen Platform) 9

3.3.2 Phase 2: HELSB Application Initiation (VeriScreen Platform) . . 11

3.3.3 Phase 3: Student HELSB Portal Access 11

3.3.4 Phase 4: Payment Processing 11

3.3.5 Phase 5: Automated Screening 11

3.3.6 Phase 6: Contract Generation and Delivery 12

3.3.7 Phase 7: Loan Disbursement 12

3.4 Data Collection 12

3.5 Tools and Technologies 13

3.5.1 Programming Languages: 13

3.5.2 Frameworks and Libraries: 13

3.5.3 Database: 15

3.5.4 Development Environment: 15
3.5.5 Additional Tools: 15
3.6 Ethical and Legal Considerations 15
3.6.1 Data Privacy: 15
3.6.2 User Consent: 16
3.6.3 Security Concerns: 16
3.6.4 Intellectual Property: 16
3.6.5 Simulated Data Ethics: 16
3.6.6 Notification Simulation: 17

4 Expected Outcomes **17**
4.1 Software Systems / Prototypes: 17
4.2 Document Generation Engine: 17
4.3 Documentation: 18
4.4 Demonstration Materials: 18
4.5 Research Contributions: 18

5 Project Timeline **19**

6 Risk Analysis **20**

7 References **23**

1 Introduction

In Zambia's higher education landscape, students frequently require official academic documents for various purposes, including loan applications, scholarship opportunities, employment, and further studies. Documents such as confirmation slips, examination results, fee payment histories, and academic status letters serve as essential proof of a student's standing within their institution. Currently, students must obtain these documents through manual processes—visiting university offices, waiting in queues, and paying for physical stamps—each time an official copy is needed.

Simultaneously, institutions like the Higher Education Loans and Scholarships Board (HELSB) require verified versions of these documents to process student loan applications. The current workflow creates a cycle of inefficiency: students download documents from university portals, upload them to HELSB platforms, and then HELSB staff must manually verify each document's authenticity with the originating university. This manual verification process introduces significant delays in loan disbursement and creates vulnerabilities for document forgery and fraud.

The motivation for this project stems from the observation that student data already exists in digital form within university databases. The missing element is a secure, trusted bridge that allows this verified data to flow directly to students and authorized external institutions. VeriScreen aims to fill this gap by creating a platform that serves two interconnected purposes: empowering students with on-demand access to their verified academic documents, and providing an automated channel for submitting these verified records to HELSB.

This project is important for several reasons. First, it addresses the growing need for digital transformation in Zambia's public service sector, particularly in education financing. Second, it tackles the persistent problem of document fraud, which undermines trust in academic credentials. Third, it reduces administrative burden on university staff who currently spend hours manually stamping documents. Fourth, it accelerates loan processing times, enabling students to access funding more quickly. Finally, it aligns with global trends toward user-controlled data ownership, where individuals have direct access to and control over their personal information.

By creating VeriScreen, this project contributes a practical solution to a real-world problem affecting thousands of Zambian students annually, while demonstrating the technical feasibility of secure inter-institutional data exchange.

1.1 Problem Statement

Students in Zambia face two interconnected challenges when dealing with academic documents for loan applications:

1.1.1 Challenge 1: Document Accessibility

Currently, when students need official academic documents, whether for HELSB applications, job interviews, or scholarship opportunities, they must physically visit university administrative offices with printed copy or copies and wait for manual stamping. This process is time-consuming, inconvenient as there are long queues, and particularly difficult for distance learners or students located far from their institutions. Despite the fact that all this information exists digitally in university databases, students have no direct, self-service mechanism to access their own verified documents.

1.1.2 Challenge 2: Loan Application Verification

When applying to HELSB, students must:

1. Download documents (confirmation slips, payment histories, exam results) from their university portals
2. Re-upload the same documents to the HELSB application platform
3. Wait for HELSB staff to manually verify each document's authenticity with the university

This creates a cascade of problems:

- **Inefficiency:** Students perform redundant download/upload cycles, wasting time and effort
- **Verification Delays:** Manual verification can take weeks, delaying loan disbursement and allowances
- **Fraud Vulnerability:** Documents can be altered or forged, leading to fraudulent applications
- **Administrative Burden:** University staff spend hours daily verifying documents for students so that they can proceed with their HELSB application
- **Data Inconsistency:** Discrepancies may exist between uploaded documents and official records

- **Poor User Experience:** Students face stress and uncertainty about application status

The core problem is the absence of a trusted digital channel that serves both students' need for document access and institutions' need for verified data.

1.2 Project Aim

To design and develop VeriScreen, a secure digital platform that bridges university databases with students and external authorities, enabling on-demand access to verified academic documents and automated submission to loan agencies like the Higher Education Loans and Scholarships Board (HELSB).

1.3 Objectives

1. To design a secure system architecture that facilitates API-based communication between a university database, student users, and external institutions like HELSB.
2. To develop a user-friendly web application where students can authenticate, view, and download verified academic documents including confirmation slips, exam slips, payment histories, and academic results.
3. To implement an automated "Submit to HELSB" feature that packages verified student data and transmits it securely to the HELSB system upon student consent.
4. To create a verification mechanism that generates digitally sealed documents with unique identifiers and timestamps, ensuring authenticity for any recipient.
5. To design a complementary HELSB portal where students can confirm their information, make payments, receive loan contracts, and where officers can manage applications.

1.4 Scope of the Project

1.4.1 In Scope:

- Development of a prototype web application for students (responsive design)
- Student authentication integration with a university database. **Preference will be given to connecting to a real university database if access is granted; otherwise, a simulated database with realistic mock student records will be used.**

- Document viewing and download functionality for:
  - Confirmation/registration slips
  - Examination entry slips
  - Fee payment histories
  - Academic results/transcripts
- "Submit to HELSB" feature that sends verified student data packages
- Development of a backend API server handling all business logic
- Development of a **fully functional HELSB screening portal** that will be used to:
  - Allow students to log in using generated credentials
  - Display a pre-filled information confirmation screen
  - Process simulated payments
  - Collect bank details
  - Perform automated eligibility screening based on verified academic data
  - Generate and deliver loan contracts via email/SMS
- Development of a simulated HELSB officer portal for application management
- Generation of digitally verified documents with:
  - QR codes or unique verification IDs
  - Timestamps and digital seals
  - Tamper-evident features
- Email and SMS notification simulation
- Basic security implementation (authentication, encryption, access control)
- Audit logging of all document accesses and submissions

1.4.2 Out of Scope:

- Full integration with live production university systems (e.g., Banner, Oracle Campus Solutions) if API access is not provided
- Integration with the actual HELSB production system

- Comprehensive administrative dashboard for university staff management
- Live payment gateway integration (simulated only)
- Support for multiple external institutions beyond HELSB
- Document editing or modification features
- Mobile app development (web application only)
- Biometric authentication features

2 Literature Overview

This project draws upon established research and technologies in e-governance, system integration, digital verification, and user-centric design.

E-Government and Digital Public Services: Heeks (2006) argues that e-government initiatives succeed when they address real user needs and integrate seamlessly with existing systems. VeriScreen aligns with this principle by targeting a specific pain point—academic document verification—and designing for integration with existing university databases. The Zambian government's broader digitalization agenda provides additional context for why such a system is timely and relevant [5].

Digital Document Verification Technologies: Stallings (2017) provides comprehensive coverage of cryptographic techniques used to ensure document integrity and authenticity. Digital signatures, hash functions, and public key infrastructure (PKI) form the technical foundation for creating tamper-evident documents. VeriScreen will implement similar principles, though scaled appropriately for a prototype [8].

API-Centric System Integration: Richardson and Amundsen (2013) establish RESTful APIs as the modern standard for secure, scalable system integration. Their work on API design patterns directly informs how VeriScreen will interface with both university databases and HELSB systems. The use of well-documented, stateless APIs ensures that each component remains loosely coupled and maintainable [7].

User Data Ownership and Consent: Hardjono and Pentland (2019) introduce the concept of data cooperatives, where individuals gain control over their personal data through trusted intermediaries. VeriScreen embodies this philosophy by giving students direct access to their academic data and requiring explicit consent before any data sharing with HELSB. This user-centric approach builds trust and aligns with emerging data protection norms [4].

Data Privacy Regulations: The European Union's General Data Protection Regulation (GDPR) (2016) establishes principles of data minimization, purpose limitation, and user consent that have become global benchmarks for ethical data handling. While Zambia has its own Data Protection Act (2021), the GDPR framework provides useful guidance for designing systems that respect user privacy. VeriScreen's consent-based design reflects these principles [1].

Local Context: Higher Education Financing in Zambia: The Higher Education Loans and Scholarships Board Act (2016) establishes HELSB's mandate to administer student loans. Reports from the Ministry of Education have highlighted delays in loan disbursement partly attributable to manual verification processes. This local context grounds the project in a real-world problem affecting Zambian students [2].

3 Proposed Methodology

3.1 Development Approach

The project will be developed using an **Agile methodology**, specifically a modified Scrum approach suitable for a team of two developers. This approach is selected because:

- It allows for iterative development with continuous refinement
- Features can be prioritized and delivered incrementally
- Regular testing ensures integration points work correctly
- Feedback from informal user testing can be incorporated

The development will proceed in two-week sprints, with each sprint delivering a working increment of the system. Sprint reviews will assess progress against objectives, and the backlog will be adjusted as needed.

3.2 System Architecture

The VeriScreen system will employ a three-tier architecture:

3.3 Complete Workflow Description

3.3.1 Phase 1: Student Document Access (VeriScreen Platform)

1. **Student Registration/Login:** Student visits VeriScreen web application and logs in using Student ID and password authenticated against the university database.

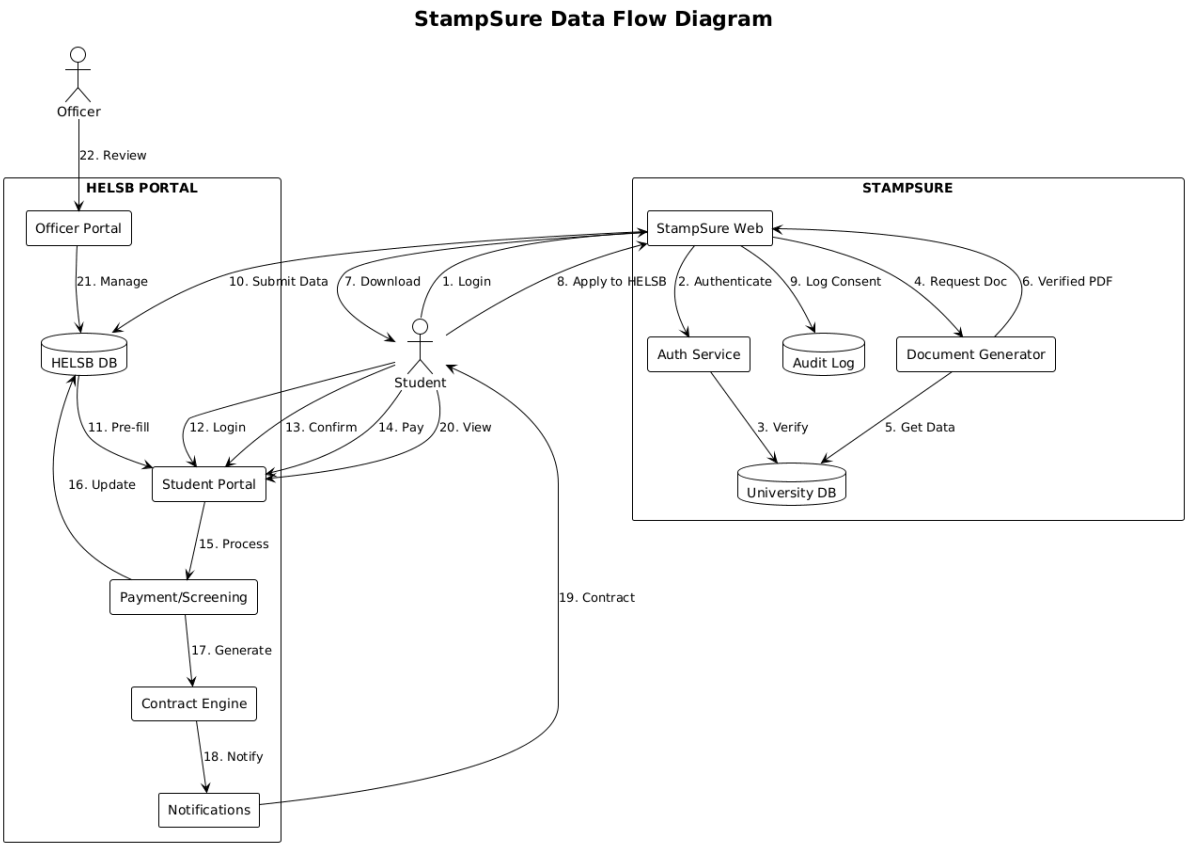


Figure 1: VeriScreen System Architecture

2. **Document Dashboard:** Upon successful authentication, student sees available document categories including confirmation slips, examination slips, fee payment histories, and academic results.

3. **Document Request:** Student selects desired document type to view or download.

4. **Data Retrieval:** Backend API fetches relevant data from the university database in real-time. If a live connection is unavailable, data will be retrieved from a simulated database.

5. **Document Generation:** System generates a digitally verified PDF document with university branding, student details, QR code, digital timestamp, and unique verification ID.

6. **Student Action:** Student can view the document, download it for personal use, or proceed to HELSB application.

3.3.2 Phase 2: HELSB Application Initiation (VeriScreen Platform)

7. **HELSB Application Initiation:** Student clicks "Apply to HELSB" button and reviews documents to be shared.
8. **Consent Provision:** Student provides explicit consent for data sharing with HELSB. System records consent with timestamp and IP address.
9. **Data Package Creation:** System packages all verified documents into a secure data bundle including personal details, academic records, and consent record.
10. **Secure Transmission:** System encrypts and transmits data package to HELSB API endpoint. Student receives confirmation email.

3.3.3 Phase 3: Student HELSB Portal Access

11. **Credential Generation:** Student receives email/SMS with HELSB portal link and temporary password.
12. **Portal Login:** Student logs into HELSB web portal using student number and temporary password.
13. **Information Review:** Student views pre-filled application with personal details and verified academic documents.
14. **Information Confirmation:** Student confirms that all information is accurate.

3.3.4 Phase 4: Payment Processing

15. **Payment Processing:** Student views application fee amount and completes simulated payment.
16. **Bank Details Entry:** Student enters banking information for loan disbursement.
17. **Payment Verification:** System verifies payment and updates status.

3.3.5 Phase 5: Automated Screening

18. **Automated Screening:** System runs eligibility checks based on academic standing, fee payment history, and registration status.
19. **Screening Results:** System updates application status (approved, under review, or not eligible).

3.3.6 Phase 6: Contract Generation and Delivery

20. **Contract Generation:** For approved students, system generates loan contract with terms and conditions.
21. **Contract Delivery:** Student receives contract via email and SMS notification. Contract is also available for download on the portal.
22. **Contract Acceptance:** Student may electronically accept contract terms.

3.3.7 Phase 7: Loan Disbursement

23. **Disbursement Processing:** System simulates loan disbursement to university and student account.
24. **Final Confirmation:** Student receives final confirmation via email and SMS.

3.4 Data Collection

The project will primarily seek to integrate with a **real university database** to demonstrate the platform's viability with live data. If access to the production database is granted by the university, the system will connect to it securely via API or direct database connection (with read-only privileges) to fetch student records.

If access is not guaranteed or permitted, the project will fall back to using a **simulated dataset** designed to realistically represent university student records. This simulated dataset will include:

- **Student table:** Student ID, name, NRC, date of birth, program of study, enrollment date
- **Registration table:** Academic year, semester, confirmation status, registration dates
- **Fees table:** Payment dates, amounts, receipt numbers, outstanding balances
- **Examinations table:** Exam eligibility, sitting status, results
- **Academic results table:** Course codes, grades, grade points, GPA calculations
- **HELSB applications table:** Application status, payment status, screening results, contract details

The simulated data will include varied scenarios:

- Students with complete fee payment
- Students with partial payment
- Students eligible for exams
- Students with various academic standings
- Multiple academic years of history
- Various loan application statuses

3.5 Tools and Technologies

3.5.1 Programming Languages:

- **JavaScript/TypeScript:** For frontend web application development
- **Python:** For backend API development
- **HTML5/CSS3:** For structure and styling

3.5.2 Frameworks and Libraries:

- **Frontend (Web Application):**
  - React.js or Vue.js for component-based UI development
  - Axios for API communication
  - React Router / Vue Router for navigation
  - Tailwind CSS or Bootstrap for responsive design
  - Redux/Context API for state management (if needed)
- **Backend (Python):**
  - Django or FastAPI for REST API development
  - Django REST Framework (if using Django)
  - SQLAlchemy for database ORM
  - PyJWT for JWT token handling
  - ReportLab or WeasyPrint for PDF generation
  - python-qrcode for QR code generation
  - Cryptography library for encryption utilities

OR

- **Backend (Java):**
  - **Spring Boot:** For REST API development and application framework
    - Spring Web for building RESTful APIs
    - Spring Security for authentication and authorization
    - Spring Data JPA for database access
    - Spring Boot DevTools for development
  - **JWT Libraries:**
    - java-jwt or jjwt for JWT token creation and validation
  - **Database Connectivity:**
    - Hibernate as the ORM framework (default in Spring Data JPA)
    - JDBC for direct database connections if needed
  - **PDF Generation:**
    - iText or OpenPDF for PDF document creation
    - Apache PDFBox as an alternative
  - **QR Code Generation:**
    - ZXing (Zebra Crossing) library for QR code generation
  - **Encryption and Security:**
    - Java Cryptography Architecture (JCA) built into Java
    - Bouncy Castle for additional cryptographic algorithms
    - Spring Security Crypto for password encoding (BCrypt)
  - **Additional Utilities:**
    - Lombok to reduce boilerplate code
    - ModelMapper or MapStruct for object mapping
    - Logback or SLF4J for logging
- **HELSB Portal (Simulated):**
  - Same frontend framework as VeriScreen
  - Separate routes/module for HELSB functionality

3.5.3 Database:

- **PostgreSQL:** Primary database for VeriScreen's own data (user sessions, audit logs, consent records)
- **MySQL or PostgreSQL:** For simulated university database (if live connection is unavailable)
- **Redis:** Optional for caching and session management

3.5.4 Development Environment:

- Visual Studio Code or PyCharm
- Postman for API testing
- Git for version control
- GitHub/GitLab for repository hosting
- DBeaver or pgAdmin for database management
- Docker for containerization (optional)

3.5.5 Additional Tools:

- Swagger/OpenAPI for API documentation
- Draw.io or Lucidchart for architecture diagrams
- Trello or GitHub Projects for sprint tracking
- Figma for UI/UX design mockups

3.6 Ethical and Legal Considerations

3.6.1 Data Privacy:

The VeriScreen system is designed with privacy as a foundational principle. Student data will only be accessed, displayed, or shared based on explicit user actions. No data is passively collected or monitored. If a live university database is used, strict adherence to the university's data protection policies and the Zambian Data Protection Act (2021) will be followed. If simulated data is used, it will contain no real student information, and any test data will be clearly fictional.

3.6.2 User Consent:

Informed consent is central to the HELSB submission feature. Before any data is transmitted to the HELSB system, the student will:

- See exactly which documents will be shared
- Be informed about how the data will be used
- Explicitly click a confirmation button to proceed
- Receive a record of their consent (digital receipt)

3.6.3 Security Concerns:

Several security measures will be implemented:

- All API communications over HTTPS
- JWT-based authentication with appropriate expiration
- Password hashing using bcrypt or similar
- Input validation and sanitization to prevent injection attacks
- Principle of least privilege for all system components
- Audit logging of all sensitive operations

3.6.4 Intellectual Property:

All code written for this project will be original. Any open-source libraries used will be:

- Properly attributed in documentation
- Used in compliance with their licenses (MIT, Apache, BSD, etc.)
- Listed with versions in a dependencies file

3.6.5 Simulated Data Ethics:

If a simulated university database is used, it will contain entirely fictional student data. No real student records will be used at any point. Names, NRC numbers, and academic details will be clearly artificial to avoid any resemblance to real individuals.

3.6.6 Notification Simulation:

Email and SMS notifications will be simulated using development tools. No real phone numbers or email addresses will be used without explicit consent from testers.

4 Expected Outcomes

At the conclusion of this project, the following deliverables will be produced:

4.1 Software Systems / Prototypes:

- **VeriScreen Student Web Application:** A responsive web application allowing students to authenticate, view, and download verified academic documents, plus submit data to HELSB.
- **VeriScreen Backend API Server:** A RESTful API handling authentication, data retrieval, document generation, and secure transmission.
- **University Database Integration:** Connection to a real university database (if access is granted) or a simulated database with realistic mock student records for testing.
- **HELSB Screening Portal:** A fully functional web interface for the loan screening process where students can confirm information, make simulated payments, enter bank details, and receive loan contracts based on automated eligibility screening.
- **Simulated HELSB Officer Portal:** A web interface for managing applications and viewing student data.
- **Email/SMS Notification System:** Simulated notification service for application updates.

4.2 Document Generation Engine:

- PDF generation module capable of creating:
  - Verified academic documents with QR codes
  - Loan contracts with terms and conditions

4.3 Documentation:

- **Technical Documentation:**
  - Complete API specification (OpenAPI/Swagger)
  - Database schema diagrams
  - System architecture documentation
  - Deployment guide
- **User Documentation:**
  - Student web application user manual
  - HELSB portal user guide (student and officer)
- **Project Report:**
  - Comprehensive final project report covering all phases

4.4 Demonstration Materials:

- Screen recording of complete end-to-end student journey
- Screen recording of HELSB officer workflow
- Test accounts with sample data for demonstration
- Presentation slides for project defense

4.5 Research Contributions:

- Analysis of challenges in academic document verification in Zambia
- Evaluation of the prototype's effectiveness
- Recommendations for future implementation and scaling

5 Project Timeline

Phase	Duration	Activities
Proposal Development	Week 1–2	Refine project concept, develop proposal document, identify initial requirements
Literature Review & System Design	Week 3–5	Review relevant literature, design system architecture, create database schemas, design API endpoints, create UI mockups in Figma
Sprint 1: Core Backend	Week 6–7	Set up development environment, implement basic API, create simulated university database, implement authentication
Sprint 2: VeriScreen Web App	Week 8–9	Develop web application login, dashboard, document viewing and download functionality
Sprint 3: Document Generation	Week 10–11	Implement PDF generation, QR code creation, digital verification features, "Submit to HELSB" feature
Midway Evaluation	Week 11	Review progress against objectives, adjust plan as needed
Sprint 4: HELSB Portal Development	Week 12–13	Develop HELSB student portal with login, confirmation, payment simulation, bank details collection, automated screening simulation
Sprint 5: Contract Generation & Notifications	Week 14	Implement contract generation, email/SMS notification simulation, officer portal
Sprint 6: Testing & Refinement	Week 15	Comprehensive testing, bug fixes, performance optimization, user experience refinements
Documentation & Report Writing	Week 16	Complete technical documentation, user manuals, final project report
Final Submission & Presentation	Week 17	Prepare final deliverables, create presentation slides, project defense

6 Risk Analysis

Risk	Description	Impact	Mitigation Strategy
Technical: Integration Complexity	Simulating realistic university and HELSB APIs may be more complex than anticipated	Medium	Begin designing mock systems early. Use well-documented API design patterns. Build incrementally, testing each integration point separately
Technical: Security Vulnerability	A flaw in authentication or data transfer could expose sensitive (simulated) data	High	Implement security best practices from the start. Use established, well-vetted libraries. Conduct code reviews and basic penetration testing. Never use real data
Project: Scope Creep	Adding features beyond the defined scope could delay completion	Medium	Strictly adhere to defined scope. Use Agile approach to prioritize core functionalities. Document "nice-to-have" features for future work
Data: Quality of Simulated Data	If mock data is not realistic, it may not properly test the system's logic	Low	Carefully design database schema based on actual university systems. Populate with varied data scenarios. Consult with current students about real document formats

Continued on next page

Risk	Description	Impact	Mitigation Strategy
Technical: PDF Generation Complexity	Creating properly formatted, verified PDFs may require more effort than expected	Medium	Research PDF libraries early. Start with simple document templates. Implement one document type at a time. Use existing templates as references
Usability: Complex Workflow	Users might find the multi-platform process confusing	High	Conduct informal user testing with fellow students. Create clear user guides. Provide tooltips and instructions throughout the applications
Technical: Web Application Compatibility	Browser compatibility issues across different devices	Medium	Use responsive design frameworks. Test on multiple browsers (Chrome, Firefox, Safari). Ensure mobile-friendly design
External: Relevance to Real Systems	The prototype might be too simplified to demonstrate value to real institutions	Medium	Research actual university and HELSB processes. Design integration logic to be conceptually accurate. Document how the prototype would connect to real systems
Personal: Time Management	Balancing project work with other academic commitments	Medium	Follow the timeline strictly. Set weekly goals. Address blockers early. Communicate with supervisor if delays occur

Continued on next page

Risk	Description	Impact	Mitigation Strategy
Technical: Data Loss	Accidental loss of code or database during development	High	Use Git for version control with regular commits. Back up database regularly. Document setup procedures to enable recovery
Technical: Email/SMS Simulation	Simulating notifications without using real contact information	Low	Use development email tools like Mailtrap. Store notifications in database for demonstration purposes

7 References

References

[1] European Parliament and Council. (2016). *Regulation (EU) 2016/679 (General Data Protection Regulation)*. Official Journal of the European Union.

[2] Government of the Republic of Zambia. (2016). *Higher Education Loans and Scholarships Board Act*. Ministry of Education.

[3] Government of the Republic of Zambia. (2021). *Data Protection Act*. Ministry of Justice.

[4] Hardjono, T., & Pentland, A. (2019). Data Cooperatives: Towards a Foundation for Decentralized Personal Data Management. *MIT Connection Science*.

[5] Heeks, R. (2006). *Implementing and Managing eGovernment: An International Text*. Sage Publications.

[6] Ministry of Education, Zambia. (2020). *Education Statistical Bulletin*. Government Printers.

[7] Richardson, L., & Amundsen, M. (2013). *RESTful Web APIs*. O'Reilly Media.

[8] Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice* (7th ed.). Pearson.

[9] Zambia Information and Communications Technology Authority. (2022). *National ICT Policy*. Government Printers.

Supervisor Approval

Name: Mrs Monica Monde Kabemba

Signature: _____

Date: March 16, 2026